

Microprogrammed Control Unit Design

Lecture Notes — Week 7

PCC CS-401: Computer Organization and Architecture

August 7, 2026

7.1 Introduction

Week 6 built a hardwired control unit for the running $R1 \leftarrow R2+R3$ example: a sequence counter (SC) plus a step decoder generated the control steps $T_1, T_2, \dots$ automatically, an opcode decoder produced one-hot D_i lines identifying the current instruction, and every control signal was realized as an OR, across instructions, of $(D_i \cdot T_j)$ AND terms — wired, physically, as gates on the chip. That design worked, but it left a specific cost exposed: adding one instruction to the machine meant re-deriving Boolean equations for every signal that instruction touches, and physically rewiring gates to match. At the scale of a real instruction set architecture — dozens to hundreds of instructions — that design and verification cost explodes [3].

This chapter asks a different question about the same problem. Week 6’s state table (step $\rightarrow$ RTL $\rightarrow$ active signals) was already a complete specification of the control unit’s behavior; two slides’ worth of work then turned that table into AND/OR gates. Was that translation necessary? A table is exactly the kind of object hardware is good at storing and reading back — that is what memory does. **Microprogrammed control** stores the state table directly, as data, and reads one row per cycle instead of evaluating gates. The idea dates to Maurice Wilkes (1951), well before instruction sets grew large enough to need it.

This chapter follows one thread through three questions, all applied to the identical $R1 \leftarrow R2+R3$ control unit from Weeks 5–6, so the hardwired-versus-microprogrammed comparison stays concrete rather than a list of adjectives: what does it mean to *store* a control step instead of wiring it (Section 7.2)? If control words live in memory, what decides which one comes next (Section 7.3)? And how wide does a stored control word need to be, and what does narrowing it cost (Section 7.4)? Section 7.5 assembles the pieces into a full microprogrammed control unit and compares it head-to-head against Week 6’s hardwired design.

7.2 Microinstructions and Control Memory

7.2.1 From State Table to Stored Data

Storing the state table as data, rather than compiling it into gates, requires two new pieces of hardware and two new pieces of notation, developed in this section.

Definition (Control Word (Microinstruction)). A **control word** is one row of stored control information: which control signals to assert this cycle, plus enough information to determine which control word to read next. A **microprogram** is the ordered sequence of control words that implements one machine instruction.

Compared to Week 6, nothing about *what* is specified changes — one row of the hardwired state table becomes one control word. The only thing that changes is whether that row is wired into gates or written

into memory. This chapter also adds one new piece of notation to the course registry: the **microaddress**, the address of a control word inside this memory.

Definition (Control Memory (CM) and Microprogram Counter (MPC)). *Control memory (CM) is a small ROM/RAM whose rows are control words, indexed by microaddress. The microprogram counter (MPC) is a register holding the microaddress of the control word being read this cycle — the microprogrammed analogue of Week 6’s SC.*

MPC feeds CM with the microaddress to read; CM’s output row — the control word — splits into two parts: the control signals that drive the datapath this cycle, and a next-address field that feeds back into MPC to determine the following cycle’s address (Figure 7.1). This is the direct microprogrammed analogue of Week 6’s SC → step decoder → control matrix pipeline: MPC plays SC’s role, and CM plays the control matrix’s role, except that CM’s output is read from memory rather than computed by gates.

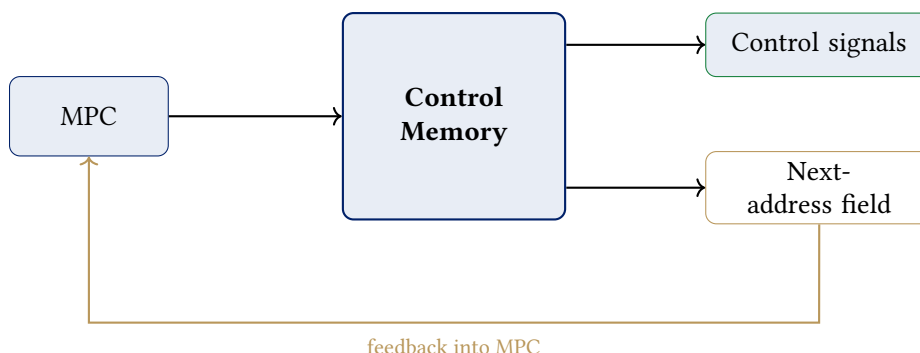


Figure 7.1: MPC addresses control memory; the control word it reads back splits into control signals (to the datapath) and a next-address field (feeding back into MPC) — cf. Mano, *Computer System Architecture*, 3rd ed., Sec. 7-1, Control Memory, p.213 [2].

Example 7.2.1 (The Microprogram for $R1 \leftarrow R2+R3$). *Rebuilding Week 5–6’s running instruction as a microprogram gives CM exactly three rows:*

Microaddress	RTL	Control word (signals asserted)	Next
0	$Y \leftarrow [R2]$	$R2_{out}, Y_{in}$	1
1	$Z \leftarrow [Y] + [R3]$	$R3_{out}, ALU=add, Z_{in}$	2
2	$R1 \leftarrow [Z]$	$Z_{out}, R1_{in}, END$	fetch

Lined up against Week 6’s hardwired state table, this is the same three rows, the same signals, and the same order — only where they live has changed: three rows of CM instead of three rows of a truth table destined for gates.

7.3 Address Sequencing

7.3.1 Locating an Instruction’s Starting Microaddress

The worked microprogram above raises an immediate question: CM does not store just ADD’s three rows — it stores every instruction’s microprogram back to back, ADD’s three rows, SUB’s three rows, LOAD’s rows, and so on. Week 6’s SC always started an instruction at T_1 (microaddress 0 in this chapter’s language), but if MPC always started at 0, the machine would only ever run whichever instruction happens to occupy CM’s first three rows. Something must translate “the opcode in IR is ADD” into “load MPC with *this instruction’s* starting microaddress.” That translation is **address sequencing**.

Definition (Address Sequencing (Mapping)). On every instruction fetch, a small **mapping** unit converts the opcode in *IR* into the microaddress where that instruction’s microprogram begins in *CM*, and loads it into *MPC*.

Mapping is the microprogrammed counterpart of Week 6’s opcode decoder: both answer “which instruction is this?”, but where the opcode decoder produced a one-hot gate-enable line, mapping produces an *address* into memory. Once the mapped starting address loads, *MPC* advances using the next-address field carried in each control word (Section 7.2) — normally the next row, until *END* triggers a new fetch and a fresh mapping. Two sources therefore feed *MPC*: the mapping unit, on a fetch, and *CM*’s own next-address field, every other cycle (Figure 7.2). Flags let a control word’s next-address field *branch* rather than simply advance — how a conditional instruction (e.g. a compare-and-jump) selects between two successors [2].

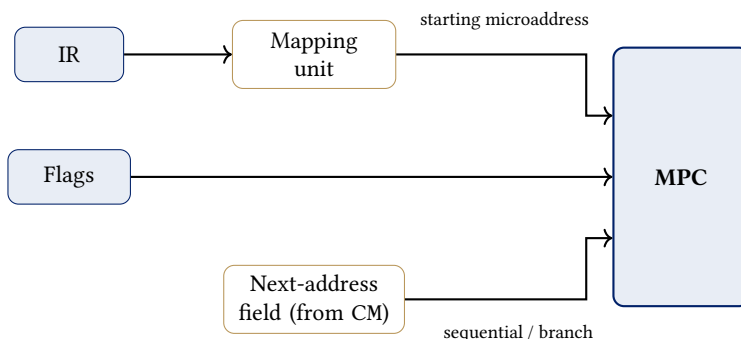


Figure 7.2: Two sources feed *MPC*: the mapping unit supplies the starting microaddress on a fetch, and *CM*’s own next-address field supplies the sequential/branch address every other cycle — cf. Mano, *Computer System Architecture*, 3rd ed., Sec. 7-2, Address Sequencing, p.215–224 [2].

Example 7.3.1 (Worked Trace Revisited: Where *MPC* Points). Laying *MPC*’s actual value alongside the RTL trace for $R1 \leftarrow R2+R3$ makes address sequencing concrete.

Step	MPC value	CM row read	RTL
Fetch	mapped $\rightarrow 0$	row 0	$Y \leftarrow [R2]$
+1	1	row 1	$Z \leftarrow [Y] + [R3]$
+1	2	row 2 (END)	$R1 \leftarrow [Z]$, then remap on next fetch

This is the piece that plays *SC*’s role: mapping plus the next-address field is what turns “read row 0, then 1, then 2” from something listed by hand into something the circuit does on its own.

7.3.2 Why Mapping Cannot Be Skipped

Suppose the mapping unit were removed and *MPC* always reset to 0 after *END*. Every instruction would then execute *CM* row 0’s microprogram — whichever instruction happens to occupy that address — regardless of what is actually in *IR*. This is exactly analogous to Week 6’s missing *SC*_{clear}: it is not optional bookkeeping. Mapping is precisely what lets a single, shared *CM* serve *every* instruction instead of just one.

7.4 Horizontal versus Vertical Encoding

7.4.1 Sizing the Control Word

The worked microprogram in Section 7.2 used about eight distinct signals ($R2_{out}$, $R3_{out}$, Y_{in} , $ALU=add$, Z_{in} , Z_{out} , $R1_{in}$, END). A real instruction set architecture has dozens to hundreds. Does every control word need one bit *per* signal, even though only a handful are ever active at once? This is a genuine width-versus-simplicity trade-off, with a name: **horizontal** versus **vertical** microinstruction encoding.

Definition (Horizontal Microinstruction). *One bit per control signal. Bit k of the control word is X_{in} (or $R2_{out}$, or END) directly — reading the word out of CM is asserting the signals, with no further logic in between.*

Horizontal encoding’s advantage is that there is no decode delay: signals are valid the instant CM is read, the same cycle. Its cost is width: the word is as wide as the number of signals in the whole machine, and most bits are 0 in any given row.

Definition (Vertical Microinstruction). *Signals that are mutually exclusive (e.g. “which register drives the bus this cycle” has exactly one answer) are grouped into a small encoded field. A **decoder** between CM and the datapath expands each field back into the actual one-hot signal.*

Vertical encoding’s advantage is a far narrower control word: n mutually-exclusive signals need only $\lceil \log_2 n \rceil$ bits, not n . Its cost is that the decoder sits on the critical path — signals are valid one decode delay *after* CM is read, not the same instant.

Example 7.4.1 (Encoding the Worked Example, Two Ways). *Row 1 of the running microprogram (T_2 : $Z \leftarrow [Y] + [R3]$, i.e. $R3_{out}$, $ALU=add$, Z_{in}) can be encoded either way.*

Horizontally, each of the eight signals gets its own bit; the three active this row ($R3_{out}$, $ALU=add$, and Z_{in}) are set to 1, the rest to 0, and reading the word out of CM directly drives the datapath (Figure 7.3).



Figure 7.3: Horizontal encoding of row 1 (T_2): 8 bits, one per signal, read directly with no decoder.

Vertically, the same row’s active signals collapse into three small fields — bus source ($=R3$), destination ($=Z$), and ALU operation ($=add$) — each needing only ≈ 2 bits, for roughly 6 bits total, but a decoder must expand these fields before the datapath sees valid signals (Figure 7.4).

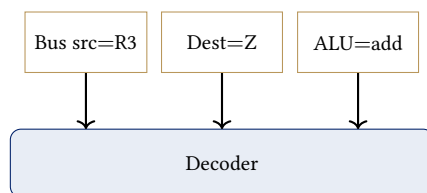


Figure 7.4: Vertical encoding of the same row: 3 fields (≈ 6 bits), needing a decode step before signals are valid.

The saving here (8 bits $\rightarrow$ ≈ 6 bits) is modest because the toy example is small — it compounds with instruction-set size, exactly where hardwired control’s design cost was exploding (Section 7.5).

7.4.2 Trade-offs

The two encodings trade width against speed in opposite directions. Horizontal encoding has no decode delay, but CM width grows linearly with the number of signals in the machine — at real-ISA scale, 100–400

bits per row is typical. Vertical encoding shrinks CM width to roughly $\log_2(\text{signals per group})$, summed across groups, but every control word pays a decoder's worth of extra delay before the datapath sees valid signals.

7.5 Synthesis: The Complete Microprogrammed Control Unit

7.5.1 Assembling the Pieces

Putting Sections 7.2–7.3 together produces the full microprogrammed control unit (Figure 7.5). On the input side, IR feeds the mapping unit, which supplies a starting microaddress; flags feed the branch decision; and the clock drives MPC's own timing — all three converge on MPC. MPC addresses CM, whose control-word output splits into control signals (to the datapath, optionally passing through a decoder if vertically encoded) and a next-address field that feeds back into MPC. This deliberately mirrors Week 6's capstone layout (IR/Flags/Clock → control matrix → outputs → feedback into SC), so the two designs read as structural twins: the same job, done by memory instead of gates.

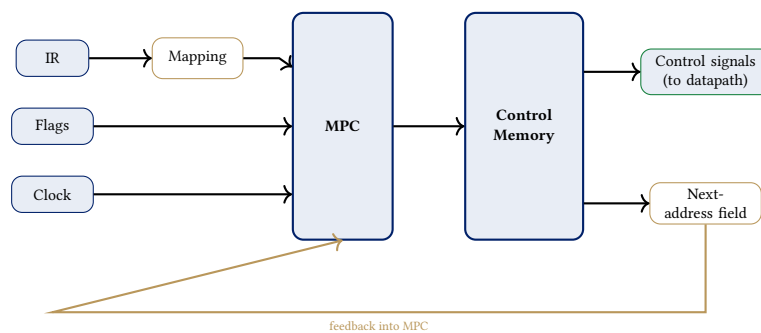


Figure 7.5: The complete microprogrammed control unit — IR/Flags/Clock feed MPC, MPC addresses CM, and CM's next-address field feeds back into MPC — cf. Mano, *Computer System Architecture*, 3rd ed., Sec. 7-2/7-4, Address Sequencing / Design of Control Unit, p.215–239 [2, 1].

Example 7.5.1 (Adding a New Instruction). Recall Week 6: adding *MUL* meant a new opcode-decoder line and rewiring an *AND* gate into an existing *OR* gate on the chip. Under microprogrammed control, the same addition costs something structurally different:

1. Write *MUL*'s control words into unused rows of CM — no new gates.
2. Extend the mapping unit with one more (opcode → starting microaddress) entry — typically itself a small memory/table, not hand-derived logic.
3. The chip's gates do not change at all; only the contents of a memory do.

7.5.2 Trade-offs of Microprogrammed Control

Microprogrammed control's advantages and costs both follow from storing control information in memory rather than wiring it into gates. On the positive side, new or modified instructions are a memory rewrite, not a redesign — even a bug fix in an already-fabricated CPU can ship as a microcode patch (real x86 chips do exactly this) — and the state table *is* the design, with no hand-derived Boolean equations to re-verify. On the negative side, every control step costs a memory read (and a decode step, if vertical), making it strictly slower per cycle than hardwired control's pure combinational logic; and CM itself is extra hardware that hardwired control does not need at all.

7.5.3 Hardwired versus Microprogrammed: Head to Head

	Hardwired (Week 6)	Microprogrammed (this chapter)
Speed per step	Fastest — pure gates	Slower — memory read (+ decode)
Adding an instruction	New/re-wired gates	New CM rows
Design method	Derive Boolean equations	Write down control words
Typical fit	Small, fixed, speed-critical ISAs (RISC)	Large, evolving ISAs (classical CISC)

Same control problem, two implementations: the choice trades raw speed against design and maintenance cost, exactly the pattern Weeks 5–6 already established for datapaths.

7.5.4 Three Threads, One Chip

Every design choice made across Weeks 5–7 has moved cost somewhere else on the chip, never eliminating it. Bus organization (Week 5): a single-bus datapath needs 3 control steps for the running trace; a three-bus datapath needs 1 — fewer steps, more wiring. Control-unit style (Weeks 6–7): hardwired is fastest per step but rigid; microprogrammed is slower per step but just a memory rewrite to change. Encoding width (this chapter): horizontal avoids decode delay but costs CM width; vertical shrinks CM but adds a decoder to the critical path. None of these is free — each trades cycle time, design effort, or raw hardware for one of the other two.

7.5.5 Summary

A **control word (microinstruction)** is one stored row of control signals plus next-address information; a **microprogram** is the ordered sequence implementing one instruction. **Control memory (CM)** plus the **microprogram counter (MPC)** replace the control matrix and SC: MPC addresses CM, whose output is the control word. **Address sequencing:** a mapping unit loads MPC with an instruction’s starting microaddress; each control word’s next-address field advances or branches MPC thereafter. **Horizontal** microinstructions (one bit per signal, no decode delay) trade against **vertical** (encoded fields, decoder required, narrower words). **Hardwired versus microprogrammed:** same control problem, opposite trade-offs — speed and rigidity versus a memory-read delay and easy modification.

Weeks 6–7 together answered “how does the CPU decide what *itself* does, every cycle?” Week 8 asks the next question: how does the CPU decide what *outside* devices — keyboards, disks, printers — do, and when? I/O devices are far slower than the CPU, so naively waiting for one wastes thousands of cycles; interrupts, polling, DMA, and I/O processors are four different answers to “who watches the device, and what happens when it’s ready?”

References

- [1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.
- [2] M. Morris Mano. *Computer System Architecture*. Prentice-Hall, 3rd edition, 1993.
- [3] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.